

The Logit Lens

Watching a prediction form, layer by layer · video 2:12

This lesson answers one question: what does a transformer “think” halfway through? The **logit lens** decodes the residual stream after every layer with the model's own final norm and unembedding. In GPT-2 you can watch “the” turn into Rome, London, then Paris, and see the last layer pull a 100%-sure “Shakespeare” back to 21%. Across real text, most decisions settle late. In Qwen2.5-0.5B the lens shows junk until the last few layers: a reminder that it is a quick, imperfect window.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 6).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. Reading every layer	▶ 0:08–0:34	1	☐
2. Watching GPT-2 decide	▶ 0:34–1:22	2	☐
3. Where the lens fails	▶ 1:22–2:01	2	☐

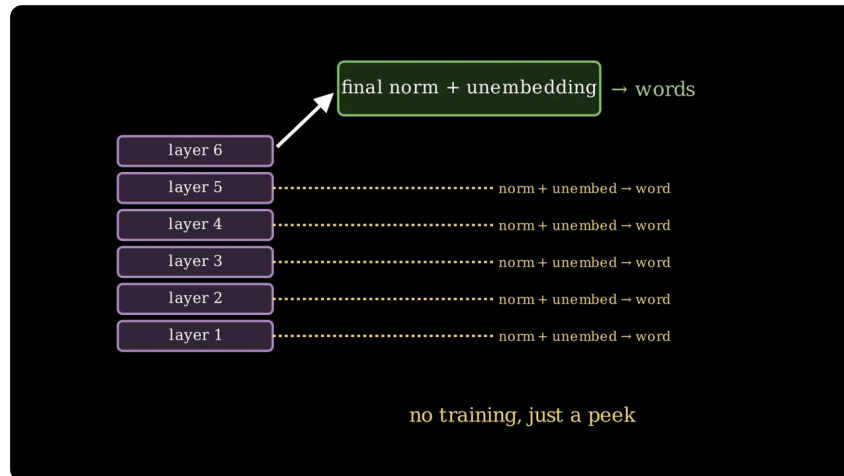
Before you start: Deep Dive episodes 13 (the residual stream) and 14 (LayerNorm vs RMSNorm).
Code: `code/d41_logit_lens` (GPT-2 small and Qwen2.5-0.5B).

CONCEPT 1 OF 3

Reading every layer

▶ Video 0:08–0:34

After each layer the residual stream holds one vector per token, and only the last one is normally turned into word probabilities, by the final norm and the unembedding matrix. The logit lens applies the same two steps to the vector after *any* layer, as if the model stopped there. No training is involved.



▶ 0:32 The final norm and unembedding, normally applied only after the last layer, applied after every layer.

KEY IDEA

Because every layer adds to the same residual stream, its intermediate states live in roughly the same space as the output, so they can be decoded.

Check 1: Reading every layer

Answer all 1 questions before looking at the answers. Pass mark: all correct.

1.1 TRUE OR FALSE

"The logit lens needs to be trained on the model's activations."

True ☐ False ☐

CONCEPT 2 OF 3

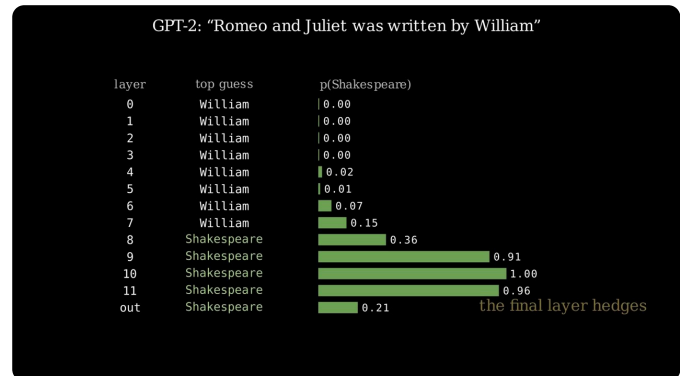
Watching GPT-2 decide

▶ Video 0:34-1:22

On “The Eiffel Tower is in the city of”, GPT-2's lens guess is filler (“the”) for most early layers, then Rome at layer 9, London at 10, and Paris from layer 11. On “Romeo and Juliet was written by William”, the early layers echo the last word; “Shakespeare” takes over at layer 8 and reaches 1.00 at layer 10, but the final layer pulls it back to 0.21. On “The capital of Japan is”, layer 11 puts “Tokyo” at 0.46 while the real output prefers “the”, with Tokyo at 0.07. The last layer hedges.

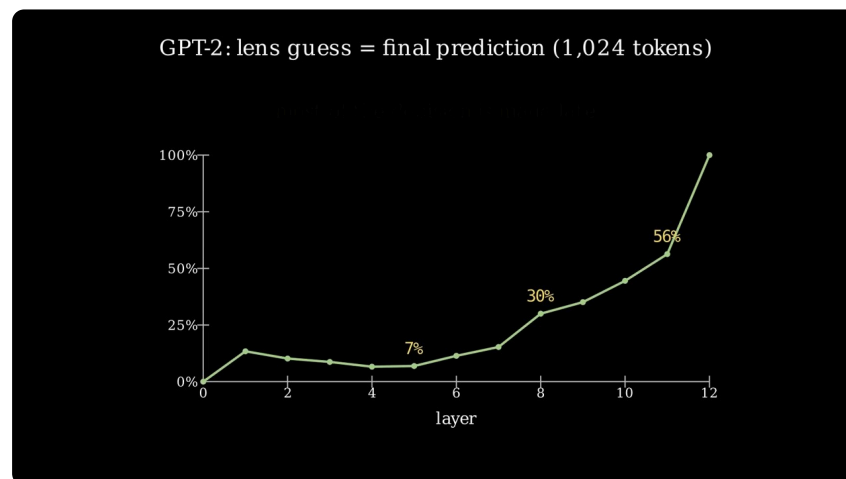


▶ 0:49 “...in the city of”: “the” for layers 1-6, then Rome (layer 9), London (10), Paris (11).



▶ 1:06 “...written by William”: “William” echoed until layer 7; Shakespeare 0.36 → 0.91 → 1.00, then 0.21 at the output.

Across 1,024 tokens of Tiny Shakespeare, the lens's top token agrees with the final prediction 7% of the time at layer 5, 30% at layer 8 and 56% at layer 11. Most of the decision is made late.



▶ 1:20 Over 1,024 tokens: lens top-1 = final prediction 7% at layer 5, 30% at 8, 56% at 11.

KEY IDEA

Predictions sharpen in the later layers; the very last layer often spreads probability out again (calibration).

Check 2: Watching GPT-2 decide

Answer all 2 questions before looking at the answers. Pass mark: all correct.

2.1 CALCULATION

In GPT-2, by how many percentage points does the lens's agreement with the final prediction grow from layer 5 to layer 11?

2.2 SHORT ANSWER

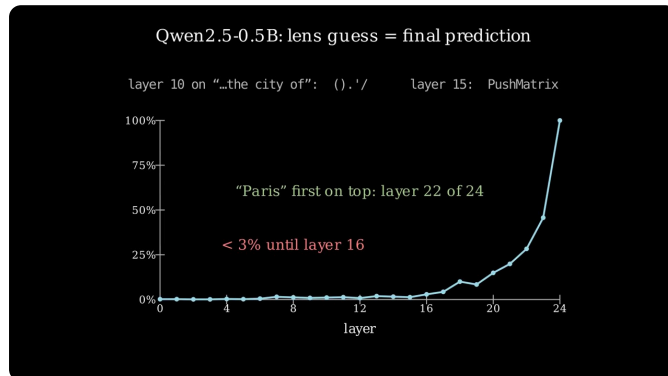
GPT-2's layer 11 gives "Tokyo" 0.46, but the output picks "the". Give a plausible reason.

CONCEPT 3 OF 3

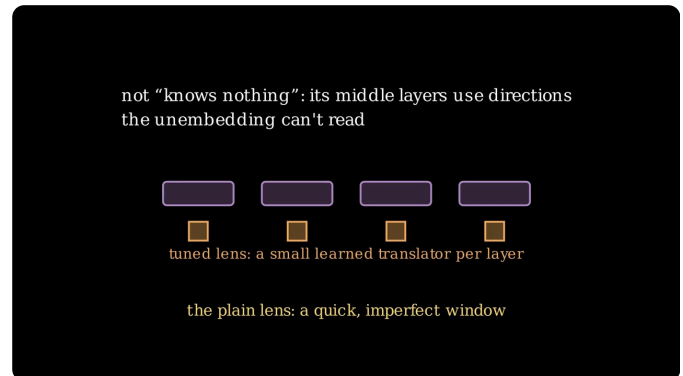
Where the lens fails

▶ Video 1:22-2:01

Qwen2.5-0.5B tells a different story: for most of its 24 layers the lens shows code fragments and pieces of words ("().'", "PushMatrix"), agreeing with the final answer under 3% of the time until layer 16; Paris only appears at layer 22. That doesn't mean Qwen knows nothing until then: its middle layers likely use directions the unembedding can't read. The **tuned lens** fixes this by training a small affine translator per layer.



▶ 1:37 Qwen2.5-0.5B: agreement below 3% until layer 16; "Paris" first on top at layer 22 of 24.



▶ 1:52 The tuned lens trains a small translator per layer.

In code the lens is one line: `model.lm_head(model.transformer.ln_f(h))` for any hidden state h .

KEY IDEA

A readable lens shows what a layer represents; an unreadable one doesn't prove the layer is empty.

Check 3: Where the lens fails

Answer all 2 questions before looking at the answers. Pass mark: all correct.

3.1 MULTIPLE CHOICE

Which conclusion does Qwen's junk lens output support?

- A. Qwen's middle layers do nothing
- B. The unembedding can't decode Qwen's middle-layer directions
- C. Qwen is a worse model than GPT-2
- D. The lens has a bug

3.2 HANDS-ON CODE

Try it yourself. Run the lens on your own prompt.

```
from transformers import GPT2LMHeadModel, GPT2TokenizerFast
tok = GPT2TokenizerFast.from_pretrained("gpt2"); model =
GPT2LMHeadModel.from_pretrained("gpt2")
ids = tok("The capital of Italy is", return_tensors="pt").input_ids
hs = model(ids, output_hidden_states=True).hidden_states
for i, h in enumerate(hs[:-1]):
    print(i, tok.decode(model.lm_head(model.transformer.ln_f(h[0, -1])).argmax()))
```

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. Reading every layer · rewatch ▶ 0:08-0:34

1.1 False. It reuses the model's own final norm and unembedding; the tuned lens is the trained variant.

2. Watching GPT-2 decide · rewatch ▶ 0:34-1:22

2.1 About 49 points (6.9% → 56.3%). Read off the agreement curve.

2.2 The last layer hedges toward continuations like “the capital ... is the city of Tokyo”, spreading probability instead of committing. The output must be calibrated over all possible continuations, not just the fact.

3. Where the lens fails · rewatch ▶ 1:22-2:01

3.1 B. Qwen still answers correctly; the information is there but not in output-readable directions.

3.2 A list of 12 guesses, mostly filler early on, with the answer appearing in the last layers. Note `hidden_states[-1]` already has the final norm applied, so it is skipped here.